# torch.stft#

https://pytorch.org/docs/stable/generated/torch.stft.html#torch.stft

**Description:**

Short-time Fourier transform (STFT). From version 1.8.0, return_complex must always be given explicitly for real inputs and return_complex=False has been deprecated. Strongly prefer return_complex=True as in a future pytorch release, this function will only return complex tensors. Note that torch.view_as_real() can be used to recover a real tensor with an extra last dimension for real and imaginary components.

## Function Signature

```
torch.stft(input, n_fft, hop_length=None, win_length=None,
window=None, center=True, pad_mode='reflect',
normalized=False, onesided=None, return_complex=None,
align_to_window=None)[source]#

Parameters

Returns

A tensor containing the STFT result with shape (B?, N, T,
C?) where

Return type
```

**Returns:**

Tensor

## Notes & Warnings

⚠ WARNING Warning From version 1.8.0, return_complex must always be given explicitly for real inputs and return_complex=False has been deprecated. Strongly prefer return_complex=True as in a future pytorch release, this function will only return complex tensors. Note that torch.view_as_real() can be used to recover a real tensor with an extra last dimension for real and imaginary components.

⚠ WARNING Warning From version 2.1, a warning will be provided if a window is not specified. In a future release, this attribute will be required. Not providing a window currently defaults to using a rectangular window, which may result in undesirable artifacts. Consider using tapered windows, such as torch.hann_window().

⚠ WARNING Warning This function changed signature at version 0.4.1. Calling with the previous signature may cause error or return incorrect result.

## Detailed Documentation

### Documentation

Short-time Fourier transform (STFT).

From version 1.8.0, return_complex must always be given explicitly for real inputs and return_complex=False has been deprecated. Strongly prefer return_complex=True as in a future pytorch release, this function will only return complex tensors.

Note that torch.view_as_real() can be used to recover a real tensor with an extra last dimension for real and imaginary components.

From version 2.1, a warning will be provided if a window is not specified. In a future release, this attribute will be required. Not providing a window currently defaults to using a rectangular window, which may result in undesirable artifacts. Consider using tapered windows, such as torch.hann_window().

The STFT computes the Fourier transform of short overlapping windows of the input. This giving frequency components of the signal as they change over time. The interface of this function is modeled after (but not a drop-in replacement for) librosa stft function.

Ignoring the optional batch dimension, this method computes the following expression:

where mmm is the index of the sliding window, and ω\omegaω is the frequency 0≤ω<n_fft0 \leq \omega < \text{n_fft}0≤ω<n_fft for onesided=False, or 0≤ω<⌊n_fft/2⌋+10 \leq \omega < \lfloor \text{n_fft} / 2 \rfloor + 10≤ω<⌊n_fft/2⌋+1 for onesided=True.

input must be either a 1-D time sequence or a 2-D batch of time sequences.

If hop_length is None (default), it is treated as equal to floor(n_fft / 4).

If win_length is None (default), it is treated as equal to n_fft.

window can be a 1-D tensor of size win_length, e.g., from torch.hann_window(). If window is None (default), it is treated as if having 111 everywhere in the window. If win_length<n_fft\text{win_length} < \text{n_fft}win_length<n_fft, window will be

padded on both sides to length n_fft before being applied.

If center is True (default), input will be padded on both sides so that the ttt-th frame is centered at time t×hop_lengtht \times \text{hop_length}t×hop_length. Otherwise, the ttt-th frame begins at time t×hop_lengtht \times \text{hop_length}t×hop_length.

pad_mode determines the padding method used on input when center is True. See torch.nn.functional.pad() for all available options. Default is "reflect".

If onesided is True (default for real input), only values for ω\omegaω in [0,1,2,…, ⌊n_fft2⌋+1]\left[0, 1, 2, \dots, \left\lfloor \frac{\text{n_fft}}{2} \right\rfloor + 1\right] [0,1,2,…,⌊2n_fft⌋+1] are returned because the real-to-complex Fourier transform satisfies the conjugate symmetry, i.e., X[m,ω]=X[m,n_fft−ω]∗X[m, \omega] = X[m, \text{n_fft} - \omega]^*X[m,ω]=X[m,n_fft−ω]∗. Note if the input or window tensors are complex, then onesided output is not possible.

If normalized is True (default is False), the function returns the normalized STFT results, i.e., multiplied by (frame_length)−0.5(\text{frame_length})^{-0.5}(frame_length)−0.5.

If return_complex is True (default if input is complex), the return is a input.dim() + 1 dimensional complex tensor. If False, the output is a input.dim() + 2 dimensional real tensor where the last dimension represents the real and imaginary components.

Returns either a complex tensor of size (∗×N×T)(* \times N \times T)(∗×N×T) if return_complex is true, or a real tensor of size (∗×N×T×2)(* \times N \times T \times 2) (∗×N×T×2). Where ∗*∗ is the optional batch size of input, NNN is the number of frequencies where STFT is applied and TTT is the total number of frames used.

This function changed signature at version 0.4.1. Calling with the previous signature may cause error or return incorrect result.

input (Tensor) – the input tensor of shape (B?, L) where B? is an optional batch

dimension

n_fft (int) – size of Fourier transform

hop_length (int, optional) – the distance between neighboring sliding window frames. Default: None (treated as equal to floor(n_fft / 4))

win_length (int, optional) – the size of window frame and STFT filter. Default: None (treated as equal to n_fft)

window (Tensor, optional) – the optional window function. Shape must be 1d and <= n_fft Default: None (treated as window of all 111 s)

center (bool, optional) – whether to pad input on both sides so that the ttt-th frame is centered at time t×hop_lengtht \times \text{hop_length}t×hop_length. Default: True

pad_mode (str, optional) – controls the padding method used when center is True. Default: "reflect"

normalized (bool, optional) – controls whether to return the normalized STFT results Default: False

onesided (bool, optional) – controls whether to return half of results to avoid redundancy for real inputs. Default: True for real input and window, False otherwise.

return_complex (bool, optional) –

whether to return a complex tensor, or a real tensor with an extra last dimension for the real and imaginary components.

Changed in version 2.0: return_complex is now a required argument for real inputs, as the default is being transitioned to True.

Deprecated since version 2.0: return_complex=False is deprecated, instead use return_complex=True Note that calling torch.view_as_real() on the output will recover the deprecated output format.

B? is an optional batch dimension from the input.

N is the number of frequency samples, (n_fft // 2) + 1 for onesided=True, or otherwise n_fft.

T is the number of frames, 1 + L // hop_length for center=True, or 1 + (L - n_fft) // hop_length otherwise.

C? is an optional length-2 dimension of real and imaginary components, present when return_complex=False.

---